Python Automata Library (PAL) Cheat Sheet

Conventions
import PythonAutomataLibrary as pal · Safe mode is default. Call **pal.FastMode()** before constructing any PAL object. Put substantive model kernels under **@pal.njit(cache=True)**. **dims** gives grid size per axis: **(nx,)** for 1D, **(nx, ny)** for 2D, **(nx, ny, nz)** for 3D. **x...** means **x**, **x,y**, or **x,y,z** in 1D/2D/3D. Negative dimensions enable wrapping; out-of-range coordinates wrap on wrapped axes and are skipped on nonwrapped axes.

Create state

pal.NewGrid(dims, dtype): Create a typed lattice storing one value per site.
pal.NewAgentGrid(dims, numAgentProps=0, isStackable=False): Create agents with optional properties; use **dims=()** for nonspatial agents.
pal.NewPopGrid(dims, capacity=None): Create integer counts per site; capacity limits total population.
pal.NewPDEgrid(dims): Create a continuous field for PDE updates.
pal.NewIList(): Create a mutable integer list for collecting/reusing query results.
pal.NewMultinomial(): Create a sampler for binomial/multinomial population draws.

Shared lattice geometry

Properties: **xDim/yDim/zDim**, **nDims**, **wrapX/Y/Z**; a negative dimension enables wrapping on that axis.
ToI(x...): Convert 1D/2D/3D coordinates into one linear site index.
ItoX/Y/Z(i): Convert a linear site index back to its axis coordinate.
grid.Hood(hood, x...): Apply relative offsets at **x...**. Create **hood** with **pal.MooreHood(dim, ...)** (diagonals), **pal.VonNeumannHood(dim, ...)** (axis-aligned), or **pal.CircleHood(dim, rad, ...)** (radius). All accept **excludeCenter=False**.

Grid / common indexing

Indexing: **g[i]** reads/writes by linear index; **g[x...]** uses dimensional coordinates.
Slices: Return detached NumPy copies, not live views; editing a slice does not update the grid.
Box(x1,x2,...): Select a rectangular region with lower and exclusive upper bounds for each axis, in axis order.

Draw / output

pal.StartPixWindow(...): Create a pixel buffer and display window; **headless=True** supports noninteractive rendering.
pix[x,y] = RGB: Set a pixel color in the shared pixel buffer.
PixWindow controls: **win.Update()** refreshes the display; **win.Save(path,block=True)** saves an image; **win.Close()** releases the window.
PixWindow GIF: **win.StartGif(path,delay=100)** starts recording; **win.AddGifFrame(block=False)** appends a frame; **win.StopGif()** finishes.
pal.StartOpenGLWindow(xDim,yDim,zDim=None,width=800,height=800,...): Create the renderer and return **(draw, win)**: **draw** collects primitives; **win** handles display, updates, and output. Omit **zDim** for 2D; set it for 3D. **headless=True** supports noninteractive rendering.
draw.Circle(rad,color,x,y,z=None): Add a circle or sphere-like primitive with the given radius and color.
draw.Box(xLen,color,x,y,z=None,yLen=None,zLen=None): Add a box centered at a position; specify axis lengths to control its size.
draw.BoxSQ(color,x,y,z=None): Add a box aligned to a lattice site.
draw.Line(width,color,x1,y1,x2,y2,z1=None,z2=None): Add a line segment between two 2D/3D points.
draw.Borders(width,color): Draw grid borders with the requested line width and color.
win.Camera(x,y,z,yaw=None,pitch=None): Set the camera position and optional yaw/pitch angles for a 3D scene.
OpenGL frame: **win.Background(color)** sets the background; **draw.Clear()** clears collected primitives; **win.Clear()** clears the scene; **win.Update()** renders.
OpenGL output: **win.IsOpen()** checks the window; **win.Save(path,block=False)** saves an image; **win.Close()** releases resources.
OpenGL GIF: **win.StartGif(...)** begins recording; **win.AddGifFrame(...)** adds a frame; **win.StopGif(...)** finishes.

Lists + randomness

IList methods: **Append(i)** adds an integer; **Clear()** empties the list; **Random()** picks an entry; **Shuffle()** reorders entries; **All()** copies the list; **Iter()** traverses without copying.
pal.Seed(seed): Seed PAL's shared random stream for reproducible call sequences.
pal.Random(): Draw a uniform random number from PAL's random stream.
pal.RandInt(n): Draw an integer from 0 through n–1; n must be positive. PAL's stream is separate from NumPy's RNG.
m.Binomial(n,p): Draw a binomial count with n trials and probability p.
m.Setup(...) / m.Sample(...): Configure a multinomial sampler, then draw counts from it.

AgentGrid

NewAgentSQ(x...) / MoveSQ(a,x...): Create or move an agent using lattice-site coordinates; documented linear-site forms are also supported.
NewAgent(x...) / Move(a,x...): Create or move an agent using continuous coordinates rather than lattice sites.
grid[a,p]: Read or write property p for agent a.
Agent position: **I(a)** gives the linear site index; **XSQ/YSQ/ZSQ(a)** give lattice coordinates; **X/Y/Z(a)** give continuous coordinates.
Alive(a) / Dispose(a): Check whether an agent is still valid, or remove it from the grid.
GetPop(): Count agents currently in the grid.
AgentsAt(x...) / LastAgent(x...): Find agents at a site or retrieve the last agent there.
AgentsInRadius(...): Find nearby agents, respecting periodic boundaries.
counts[x...]: Read site occupancy counts without modifying them.
Iteration: **agents.All()** returns a detached snapshot, safe for iteration while adding or disposing agents.
DispWrapX/Y/Z(p1,p2): Compute the shortest displacement along a periodic axis.

PopGrid + PDEgrid

Synchronous updates: **Add(v,...)** queues a change; **Update()** applies queued changes together; **Reset()** clears current and pending values. Indexed assignment changes current state immediately.
pop.GetPop(): Return total population across all sites.
pop.All(): Return indices of sites with nonzero population, not the population values.
PDE setup: **field.SetTimeSpaceStep(dt, dx[,dy,dz])** sets time and spatial steps before updates. Use spacing values that satisfy the selected scheme's stability requirements.
Diffusion methods: **Diffusion(...)** uses the standard scheme; **DiffusionMask(...)** applies a mask; **DiffusionField(...)** uses spatially varying coefficients; **DiffusionInterfaces(...)** specifies interfaces; **DiffusionADI(...)** uses an implicit scheme; **DiffusionRadialCircle/Sphere(...)** handles radial

Python Automata Library (PAL) Cheat Sheet

geometry.
Advection methods: **Advection(...)** uses constant velocity;
AdvectionField(...) uses spatially varying velocity;
AdvectionInterfaces(...) specifies interface velocities.
More detail: Manual · API Guide · API Reference (Documentation/).